CAPSTONE PROJECT

CHAPTER 4 - SYSTEM DESIGN & DEVELOPMENT

Student / Group Template

Project Title	Campus Help Desk Ticketing System: A Database-Driven Application for IT Support Request Management
Group Members	Freek Schaefer-Gerber and Wian Sandenbergh
Module	SPM622 / MDB622
Lecturer / Supervisor	Mr. Dunstan
Week / Stage	Design & Development finalized
Submission Date	29 October 2026

CTU Training Solutions (PTY) Ltd is fully accredited by the MICT SETA (ACC/2006/07/186) through the Quality Council for Trades and Occupations (QCTO). CTU Training Solutions is registered with the Department of Higher Education and Training as a Private College (2018/FE07/004) and provisionally registered as a private higher education institution (No 2014/HE07/004) Co. Reg No:2005/038944/07

4.1 Introduction

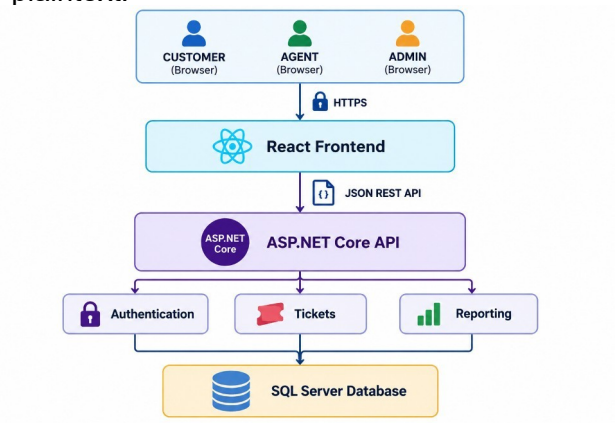
Explain the purpose of the design and implementation chapter.

This chapter presents the implemented architecture, models, database structure, interface design, development environment and implementation approach for the SmartDesk Customer Support Ticketing System. The application provides a central location where customers can raise support requests and staff can assign, respond to, track and resolve them.

4.2 System Architecture

Present and explain the architecture diagram, components and data flow.

The implemented system uses a three-layer monolithic web architecture. The presentation layer uses server-rendered Jinja2 HTML templates and CSS. The application layer is built with Python and the Flask framework, which handles routing, server-side validation, and role-based authorization. The data layer uses a relational SQLite database accessed via the sqlite3 standard library using parameterised queries. Passwords are stored using Werkzeug password hashing rather than plaintext.



Layer	Responsibility	Proposed technology
Presentation	Forms, dashboards, filters, ticket details	Jinja2 / HTML5 / CSS3
Application	API endpoints, authorization, validation, workflow rules	Python 3 / Flask
Data	Users, tickets, categories, comments, assignments and audit trail	SQLite 3
Collaboration	Source control, pull requests and change history	Git and GitHub

Illustrative request flow: (1) customer signs in; (2) customer submits a ticket; (3) API verifies identity and validates fields; (4) database stores ticket with Open status; (5) agent/admin views and assigns it; (6) agent updates status or comments; (7) customer sees the updated ticket history.

4.3 UML Design

Include and explain relevant use case, class, activity and sequence diagrams.

4.3.1 Use case model

Actor	Use cases
Customer	Register/sign in; create ticket; view own tickets; comment; view status
Support Agent	View assigned tickets; add response; update priority/status; resolve ticket
Administrator	Manage users and categories; assign tickets; view all tickets and reports

Authorization boundary: customers can only access their own tickets; agents can access assigned tickets; administrators can access all tickets. Exact privileges should be confirmed during requirements review.

4.3.2 Core class model

```
ApplicationUser (Id, FullName, Email, Role)
   1 ──< Ticket (TicketId, Subject, Description, Status, Priority, CreatedAt)
Ticket >── 1 Category (CategoryId, Name)
Ticket 1 ──< TicketComment (CommentId, Body, CreatedAt, AuthorId)
Ticket 1 ──< TicketStatusHistory (HistoryId, OldStatus, NewStatus, ChangedAt)
Ticket >── 0..1 AssignedAgent (ApplicationUser)
```

4.3.3 Ticket activity diagram (textual representation)

```
Start → Sign in → Create ticket → Validate submission
      → [Invalid] Display errors → Correct submission
      → [Valid] Save as Open → Admin assigns agent
      → Agent investigates → In Progress
      → [More information needed] Awaiting Customer → Reply
      → Agent resolves → Resolved → Close → End
```

4.3.4 Ticket creation sequence

```
Customer → Web UI: submit ticket form
Web UI → API: POST /api/tickets
API → Auth: verify identity and role
API → Database: INSERT Ticket
Database → API: ticket ID
API → Web UI: 201 Created + ticket summary
Web UI → Customer: show confirmation
```

4.4 Database Design

Present ERD, entities, attributes, keys, relationships and normalisation where applicable.

The SQLite database consists of three tables: users, tickets, and comments. The tickets table links to the users table twice: once for the customer who created the ticket (customer_id) and once for the assigned agent (assigned_to, which is nullable). The comments table links to both tickets and users to maintain an attributable conversation.

Entity	Key fields	Purpose
ApplicationUser	Id (PK), FullName, Email	Customer, agent and administrator identities
Category	CategoryId (PK), Name (unique)	Ticket categorisation
Ticket	TicketId (PK), CustomerId (FK), AssignedAgentId (nullable FK), CategoryId (FK), Subject, Description, Priority, Status, CreatedAt, UpdatedAt	Main customer support request
TicketComment	CommentId (PK), TicketId (FK), AuthorId (FK), Body, CreatedAt	Conversation associated with ticket
TicketStatusHistory	HistoryId (PK), TicketId (FK), ChangedById (FK), OldStatus, NewStatus, ChangedAt	Traceable status transitions

4.4.1 Cardinality

One customer can create many tickets (1:M). A staff member can be assigned many tickets, but a ticket has zero or one assignee (1:M optional). A ticket can contain many comments (1:M).

4.4.2 Normalisation and integrity

The schema targets 3NF. Users are not duplicated in ticket records (only customer_id and assigned_to are stored). Repeating comments are separated into the comments table. Category, priority, and status are enforced using CHECK constraints rather than lookup tables to maintain MVP simplicity.

The accompanying SQL schema is a proposed standalone relational model. If ASP.NET Core Identity is adopted, the Users table in the script must be replaced or integrated with the framework-managed AspNetUsers schema before production use.

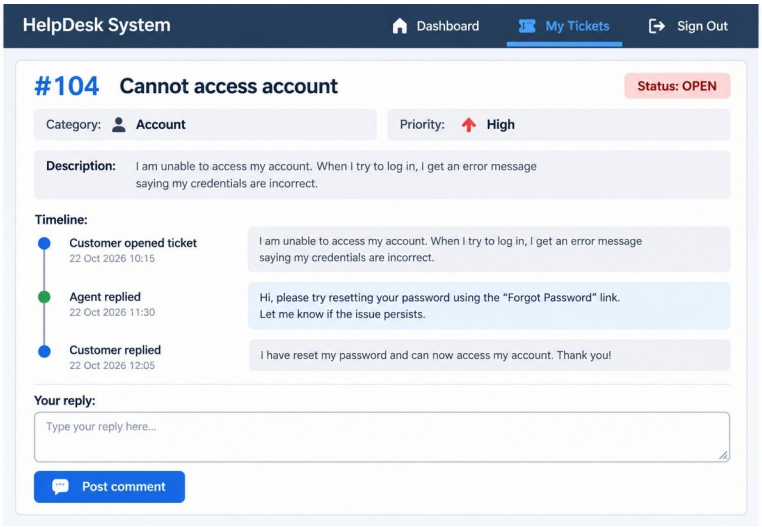
4.5 User Interface Design

Show wireframes or prototypes and explain usability decisions.

The UI should use a responsive layout with clear navigation, readable contrast, form labels, validation feedback and consistent ticket status badges. Customers should see their own ticket list and a prominent “Create Ticket” action; agents need an assigned-ticket queue; administrators need a workload dashboard and management views.

Screen	Main elements
Login/Register	Email, password, validation messages, secure session handling
Customer dashboard	My tickets, ticket count by status, create-ticket action
Create ticket	Subject, category, description, priority (if permitted), submit button
Ticket details	Status, assigned agent, chronological comments, reply field
Agent queue	Assigned tickets, search, filters, status updates
Admin dashboard	Unassigned tickets, assignment controls, category and user management, reports

Wireframe sketch — Ticket Details



4.6 Development Environment

List languages, frameworks, IDE, database, libraries, cloud/services and version-control tools.

Component	Proposed selection
Frontend	Jinja2 HTML templates, CSS3
Backend	Python 3.11+, Flask 3
Database	SQLite 3 (runtime), SQL Server 2022
IDE	VS Code / PyCharm
Testing	pytest
Version control	Git and GitHub
API testing	Swagger/OpenAPI interface and/or Postman
Deployment	Local demonstration initially; hosting only if feasible

These technologies are proposed and must be updated if the team chooses a different implementation. No paid cloud hosting is necessary for a local capstone demonstration.

4.7 Implementation

Explain important modules/features and include selected code snippets only where they support the discussion.

4.7.1 Proposed modules

Authentication and roles: register/login, session management, secure password handling, role checks. Ticket management: create, list, search, view and update tickets. Assignment: administrators assign an agent. Communication: ticket comments. Reporting: counts by status, priority and assignee, with basic resolution-time calculations.

4.7.2 API contract (planned)

Method / endpoint	Purpose	Access
POST /api/auth/register	Register account	Public
POST /api/auth/login	Authenticate	Public
GET /api/tickets	List tickets visible to current user	Authenticated
POST /api/tickets	Create ticket	Customer
GET /api/tickets/{id}	Read permitted ticket	Authenticated / authorized
PATCH /api/tickets/{id}/status	Change status	Agent/Admin
PATCH	Assign agent	Admin

/api/tickets/{id}/assign		
POST /api/tickets/{id}/comments	Add comment	Authorized ticket participant
GET /api/reports/summary	Get summary metrics	Admin

Example ticket creation payload (illustrative; the authenticated customer ID must be derived server-side, not accepted from the client):

```
{ "subject": "Cannot sign in", "description": "Login returns an error",
  "categoryId": 1, "priority": "Medium" }
```

4.7.3 Development stages and evidence

Stage 1: establish repository, API and database. Stage 2: implement authentication and CRUD. Stage 3: implement assignment, comments and role authorization. Stage 4: add dashboards and reports. Stage 5: execute tests, resolve defects, capture screenshots and finalize documentation. Evidence to insert: screenshots of login, ticket creation, ticket list, agent queue, database tables, API test output and commit history.

4.8 Version Control

Explain repository structure, branches, commits, pull/merge activity and team collaboration.

The team should use a shared GitHub repository with separate feature branches, meaningful commits and pull requests. Suggested structure: /frontend, /backend, /database, /docs, /tests. Proposed responsibilities: Freek focuses on frontend and integration; Wian focuses on backend and database, with both reviewing requirements, testing, documentation and merge requests. This division must be updated to reflect actual contributions.

Recommended evidence: repository URL [TO ADD]; screenshots of branches [TO ADD]; commit log with both authors [TO ADD]; pull request or merge history [TO ADD]. Do not report commits or completed functionality until the evidence exists.

4.9 Chapter Summary

Summarise the implemented system and prepare for testing.

This chapter defined the proposed architecture, UML interactions, relational schema, user interface, development stack and implementation sequence for the Customer Support Ticketing System. The next chapter will evaluate the working implementation through functional, integration, security and usability tests. Before submission, replace proposed design statements with verified implementation details and add actual screenshots and repository evidence.